

Fig. 17.1 Densest possible packing of jobs of τ_i , when R_i^{ub} is a safe upper bound on the response time of τ_i

17.1.1 [RTA] for Work-Conserving Schedulers

As exactly computing the effective workload executed by each interfering task within the interval of interest is hard, we seek for an upper bound of the workload that each task τ_i can execute in a generic window of length R_k . To do so, we seek to find the densest possible packing of jobs that can be generated by a legal schedule. Note that we are not relying on any particular scheduling policy, but are providing a bound that is valid for any work-conserving scheduler.

With these premises and as long as there are no deadline misses, a bound on the workload of a task τ_i in a generic interval $[a, b]$ can be determined by considering a situation in which the carry-in job begins executing right at the beginning of the interval of interest, with a response time equal to the worst-case response time of τ_i (therefore $r_i^e = a + C_i - R_i$) and every other instance of τ_i is executed as soon as possible. The situation is represented in Fig. 17.1.

As by definition of R_i , each job J_i^j executes only in $[r_i^j, r_i^j + R_i)$ and for at most C_i time units, it immediately follows that the depicted situation provides the highest possible amount of execution in interval $[a, b]$: By “sliding” the interval backwards, the carry-in cannot increase while the carry-out decreases. Similarly, if the interval were slid forward, the carry-in decreases, while the carry-out can increase by at most the same amount.

Using Fig. 17.1, we now compute the effective workload of task τ_i in an interval $[a, b]$ of length L in the situation described above.

Note that the first job of τ_i after the carry-in is released at time $a + C_i - R_i + T_i$. The next jobs are then released periodically every T_i time units. Therefore the number of body jobs within the interval is

$$\left\lfloor \frac{(a + L) - (a + C_i - R_i + T_i)}{T_i} \right\rfloor = \left\lfloor \frac{L + R_i - C_i - T_i}{T_i} \right\rfloor.$$

Adding the (full) carry-in contribution, the number $N_i(L)$ of jobs of τ_i that contribute with an entire worst-case execution time (WCET) to the workload in an interval of length L is given by $\left(\left\lfloor \frac{L + R_i - C_i - T_i}{T_i} \right\rfloor + 1 \right)$. Hence

$$N_i(L) = \left\lfloor \frac{L + R_i - C_i}{T_i} \right\rfloor. \quad (17.2)$$

The contribution of the carry-out job can then be bounded by

$$\min(C_i, (L + R_i - C_i) \bmod T_i).$$

A bound on the workload of a task τ_i in a generic interval of length L is then:

$$\widehat{W}_i(L) = \left\lfloor \frac{L + R_i - C_i}{T_i} \right\rfloor C_i + \min(C_i, (L + R_i - C_i) \bmod T_i). \quad (17.3)$$

We are now ready to state a first result valid for any work-conserving scheduler.

Theorem 17.2 *An upper bound on the response time of a task τ_k in a multiprocessor system scheduled with a work-conserving algorithm can be derived by the fixed point iteration on the value R_k^{ub} of the following expression, starting with $R_k^{\text{ub}} = C_k$:*

$$R_k^{\text{ub}} \leftarrow C_k + \left\lfloor \frac{1}{m} \sum_{i \neq k} \min(\widehat{W}_i(R_k^{\text{ub}}), R_k^{\text{ub}} - C_k + 1) \right\rfloor, \quad (17.4)$$

where $\widehat{W}_i(R_k)$ is given by Eq. 17.3.

Proof The proof is by contradiction. Suppose the iteration ends with a value $R_k^{\text{ub}} \leq D_k$, but the response time of τ_k is higher than R_k^{ub} . Since the iteration ends, it is

$$R_k^{\text{ub}} = C_k + \left\lfloor \frac{1}{m} \sum_{i \neq k} \min(\widehat{W}_i(R_k^{\text{ub}}), R_k^{\text{ub}} - C_k + 1) \right\rfloor$$

For Eq. (17.1) and $W_i(r_k, r_k + R_k) \leq \widehat{W}_i(R_k^{\text{ub}})$,

$$I_{i,k} \leq \widehat{W}_i(R_k^{\text{ub}}).$$

Therefore, as long as $R_k^{\text{ub}} \leq D_k$,

$$R_k^{\text{ub}} \geq C_k + \left\lfloor \frac{1}{m} \sum_{i \neq k} \min(I_{i,k}, R_k^{\text{ub}} - C_k + 1) \right\rfloor.$$

Since, by hypothesis, the response time of τ_k is higher than R_k^{ub} , the inverse of Theorem 17.1 gives

$$\sum_{i \neq k} \min(I_{i,k}, R_k^{\text{ub}} - C_k + 1) \geq m(R_k^{\text{ub}} - C_k + 1).$$

Therefore,

$$R_k^{\text{ub}} \geq C_k + \left\lfloor \frac{1}{m} m(R_k^{\text{ub}} - C_k + 1) \right\rfloor = R_k^{\text{ub}} + 1$$

reaching a contradiction.

It remains to show that the iteration converges in a finite amount of time. This is assured by the integer time assumption, and by noting that the assignment of Eq. (17.4) is a monotonically nondecreasing function of R_k^{ub} . $\square$

A schedulability test could be performed by repeating the iteration described above for every task $\tau_k \in \tau$. If every iteration ends before the corresponding deadline value, then the task set is schedulable. However, the problem in applying this schedulability test is that, when computing $\widehat{W}_i(R_k^{\text{ub}})$ with Eq. 17.3, the response times R_i of all interfering tasks τ_i should be known.

To sidestep this problem, we will compute Eq. 17.3 using the response time upper bounds R_i^{ub} , instead of R_i . In this respect, two possible strategies can be adopted.

With the first strategy, all response time upper bounds are initially set to $R_i^{\text{ub}} = D_i, \forall \tau_i$. Then, for each considered task τ_k , Eq. (17.4) is used to compute a new response time bound. If the computed bound is $\leq D_k$, R_k^{ub} is replaced with the new value. Otherwise, the task is temporarily set aside, continuing to the next task. If for each task τ_k , a bound $\leq D_k$ is computed, the task system is deemed schedulable. Otherwise, another round of updates is performed for all tasks using the new (smaller) response time upper bounds. When no further update is possible for any of the n tasks, and at least one task has a bound exceeding the corresponding deadline, the test fails.

Basically, when the computed response time bound of a task is $> D_k$, the test does not immediately fail, but it tries decreasing the bounds R_i^{ub} of the interfering tasks τ_i . As $\widehat{W}_i(L)$ is a nonincreasing function of the response times of the interfering tasks, using smaller response time bounds R_i^{ub} in Eq. 17.3 may result in a smaller workload $\widehat{W}_i(L)$ imposed on τ_k 's scheduling window. Equation (17.4) may then return a value $R_k^{\text{ub}} \leq D_k$. The convergence of the algorithm is guaranteed by the monotonicity of $\widehat{W}_i(L), \forall \tau_i$ with respect to L and to the response times of the interfering tasks.

A more formal version of the schedulability algorithm is shown in Fig. 17.2. Procedure `COMPUTERESPONSE`(τ_k) returns the response time upper bound R_k^{ub} computed using Theorem 17.2.

The second strategy for updating the response time upper bounds is initially setting all response time upper bounds $R_i^{\text{ub}} = C_i, \forall \tau_i$, i.e., to their minimum possible value. Then, for each task τ_k , Eq. (17.4) is used to compute a new tentative response time upper bound R_k^{ub} . If the computed bound is $> D_k$, the test fails, as there is no possibility for further improvements. Otherwise, if the new bound is larger than the previous one, R_k^{ub} is updated to the new value, passing to the next task. When an update took place, another round of updates needs to be performed for all n tasks. When no update takes place during a whole round, the test deems the task set schedulable.

This second strategy dominates the previous one, as it allows finding tighter worst-case response time upper bounds. It indeed considers a smaller (tentative) interval of interest, resulting in a possibly smaller interfering workload. However, it may require a larger run-time complexity for schedulable task systems. This is because every time there is an update in the response time bound of one of the tasks, another round of updates is triggered for the whole task system. This is not the case with the first strategy, which may deem a task set schedulable as soon as for each tasks τ_k a bound $R_k^{\text{ub}} \leq D_k$ is computed.

This second strategy is formalized in Fig. 17.3.

SCHEDCHECK(τ)

```

  ▷ Check the schedulability of a task set  $\tau$ .
  ▷ All response-time upper bounds are initially set to  $R_k^{\text{ub}} = D_k, \forall k$ ;
  ▷ Updated = true.
1  while (Updated == true) do
2    Feasible  $\leftarrow$  true
3    Updated  $\leftarrow$  false
4    for  $k = 0$  to  $n$  do
      ▷ Try to update  $R_k^{\text{ub}}$  for task  $\tau_k$ 
      NewBound  $\leftarrow$  COMPUTE RESPONSE( $\tau_k$ )
      if (NewBound  $>$   $D_k$ ) Feasible  $\leftarrow$  false
      if (NewBound  $<$   $R_k^{\text{ub}}$ )
        {  $R_k^{\text{ub}} \leftarrow$  NewBound
          Updated  $\leftarrow$  true }
    end for
    ▷ When no task is infeasible, declare success
10   if (Feasible == true) return true
  done
  ▷ When no  $R_k^{\text{ub}}$  can be updated anymore, stop the test
11 return false

```

Fig. 17.2 Response time analysis: first strategy

SCHEDCHECK(τ)

```

  ▷ Check the schedulability of a task set  $\tau$ .
  ▷ All response-time upper bounds are initially set to  $R_k^{\text{ub}} = C_k, \forall k$ ;
  ▷ Updated = true.
1  while (Updated == true) do
2    Updated  $\leftarrow$  false
3    for  $k = 0$  to  $n$  do
      ▷ Try to update  $R_k^{\text{ub}}$  for task  $\tau_k$ 
      NewBound  $\leftarrow$  COMPUTE RESPONSE( $\tau_k$ )
      if (NewBound  $>$   $D_k$ ) return false
      if (NewBound  $>$   $R_k^{\text{ub}}$ )
        {  $R_k^{\text{ub}} \leftarrow$  NewBound
          Updated  $\leftarrow$  true }
    end for
  done
  ▷ When bounds converged and no task is infeasible, declare success
9  return true

```

Fig. 17.3 Response time analysis: second strategy

17.1.2 [RTA] for EDF

When adopting an earliest-deadline-first (EDF) scheduler, a tighter upper bound on the workload executed by the interfering tasks within the interval of interest can be derived, by noting that no job can be interfered with, by jobs with later deadlines. As in Sect. 15.1, we will consider the worst-case workload produced by an interfering

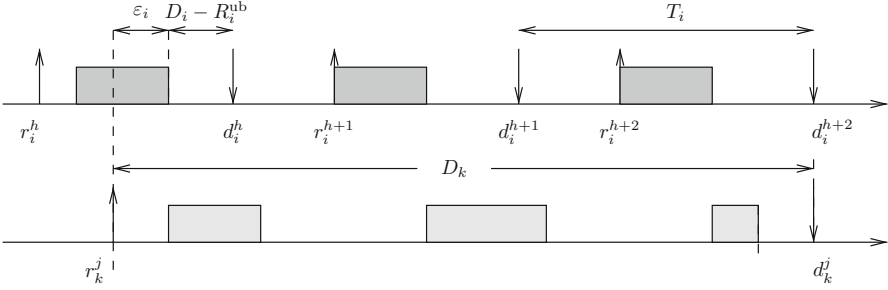


Fig. 17.4 Scenario that produces, with EDF, the maximum possible interference of task τ_i on a job of task τ_k , when R_i^{ub} is an upper bound on τ_i 's response time

task τ_i when it has an absolute deadline coincident to a deadline of τ_k , and every other instance of τ_i is executed as late as possible.

Consider the situation in Fig. 17.4. Note that the interval of interest coincides with the full scheduling window of τ_k , i.e., from the release time to the deadline of the considered job. We express τ_i 's workload separating the contributions of the first job that has a deadline inside the considered interval, from the contributions of later jobs of τ_i . There are $\left\lfloor \frac{D_k}{T_i} \right\rfloor$ later jobs, each one contributing for an entire worst-case computation time. Instead, the first job contributes for $\max(0, D_k \bmod T_i - (D_i - R_i^{ub}))$, when this term is lower than C_i . We therefore obtain the following expression:

$$I_{i,k} \leq \widehat{I}_{i,k} \doteq \left\lfloor \frac{D_k}{T_i} \right\rfloor C_i + \min(C_i, (D_k \bmod T_i - D_i + R_i^{ub})_0). \quad (17.5)$$

Note that when $R_i^{ub} = D_i$, Eq. (17.5) reduces to Eq. (15.1).

The above bound, along with the bounds presented for a general work-conserving scheduler, yields the following theorem.

Theorem 17.3 *An upper bound on the response time of a task τ_k in a multiprocessor system scheduled with global EDF can be derived by the fixed point iteration on the value R_k^{ub} of the following expression, starting with $R_k^{ub} = C_k$:*

$$R_k^{ub} \leftarrow C_k + \left\lceil \frac{1}{m} \sum_{i \neq k} \min(\widehat{W}_i(R_k^{ub}), \widehat{I}_{i,k}, R_k^{ub} - C_k + 1) \right\rceil, \quad (17.6)$$

where $\widehat{W}_i(L)$ and $\widehat{I}_{i,k}$ are given by Eq. 17.3 and 17.5, respectively.

Proof The proof is identical to that of Theorem 17.2. $\square$

The iterative schedulability test we described for general work-conserving algorithms, applies as well to the EDF case. The only difference lies at line 2 of procedure SCHEDCHECK(τ) in Fig. 17.2, where procedure COMPUTE RESPONSE(τ_k) returns the response time bound computed using Theorem 17.3.

17.1.4 Reduction of Carry-in Instances

The interval of interest adopted by the [RTA] test does not allow for limiting the number of carry-in instances to at most $m - 1$ of them, as done with the [BAR] test. Therefore, the tests are incomparable. However, it is possible to limit the number of carry-in instances to at most $m - 1$ by changing the interval of interest considered in the [RTA] analysis. Consider an interval of interest that is similar to the one adopted for the [BAR] test. Let t_o be the start of the interval, coinciding with the last time-instant prior to the arrival of the problem job of τ_k at which some processor is idle or executing a job with lower priority. Different from the [BAR] test, the end of the interval is not the deadline of the problem job, but its finishing time.

For any work-conserving scheduler, no processor is idled when there are jobs awaiting execution. There are therefore at most $(m - 1)$ carry-in jobs at the beginning of the interval. Note that from t_o to the release of the problem job, all processors are busy executing higher-priority jobs. Therefore, if the release of the problem job of τ_k is shifted left to the beginning of the interval of interest (while keeping its deadline and all previous instances of τ_k as they are), its response time cannot decrease.

An upper bound on the response time of τ_k can then be given considering a situation in which there are only $m - 1$ carry-in instances. However, since previous instances of τ_k can execute at the beginning of the interval, the interfering workload should be computed extending the sum to all tasks contributions, including τ_k 's (the [RTA] test, instead, limited the interfering workload to the tasks $\tau_{i \neq k}$, as evident from Eq. 17.6).

For each carry-in task $\tau_{i \neq k}$, the upper bound $\widehat{W}_i(L)$ on the workload in a window of length L given by Eq. 17.3 for a generic work-conserving scheduler can still be used. An upper bound on the workload executed in the interval of interest by previous jobs of τ_k can be derived by subtracting the problem job's contribution from $\widehat{W}_i(L)$. In both cases, it is sufficient to consider the amount of interfering contribution smaller than $L - C_k + 1$. Therefore, an upper bound on the carry-in workload in the interval of interest is

$$\widehat{W}_i^{CI}(L) \doteq \begin{cases} \min(\widehat{W}_i(L), L - C_k + 1), & \text{if } i \neq k \\ \min(\widehat{W}_i(L) - C_k, L - C_k + 1), & \text{if } i = k. \end{cases} \quad (17.7)$$

For tasks without carry-in, the worst-case workload is computed considering the situation in Fig. 17.5, where each interfering task is released at the beginning of the window of interest.

The number of τ_i 's jobs with both release and deadline within the interval of interest is $\lfloor L/T_i \rfloor$, and the contribution of the carry-out job is at most $\min(C_i, L \bmod T_i)$. The contribution of previous jobs of τ_k can again be derived by subtracting the problem job contribution. Considering the amount of interfering contribution smaller than $L - C_k + 1$, an upper bound on the workload of a task with no carry-in in the interval

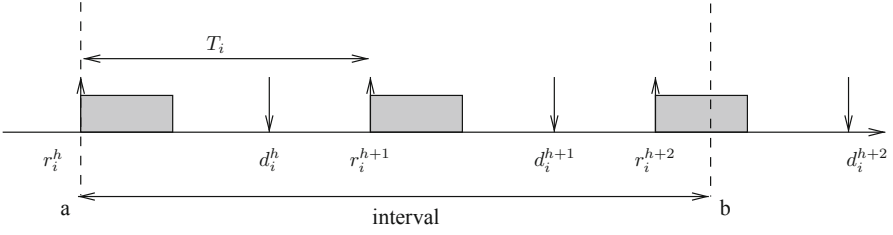


Fig. 17.5 Scenario that produces the densest possible packing of jobs of a task τ_i with no carry-in

of interest is then

$$\widehat{W}_i^{NC}(L) \doteq \begin{cases} \min \left(\left\lfloor \frac{L}{T_i} \right\rfloor C_i + \min(C_i, L \bmod T_i), L - C_k + 1 \right), & \text{if } i \neq k \\ \min \left(\left\lfloor \frac{L}{T_i} \right\rfloor C_i + \min(C_i, L \bmod T_i) - C_k, L - C_k + 1 \right), & \text{if } i = k. \end{cases} \quad (17.8)$$

Let us denote by $\widehat{W}_i^\epsilon$ the difference between $\widehat{W}_i^{CI}$ and $\widehat{W}_i^{NC}$:

$$\widehat{W}_i^\epsilon(L) \doteq \widehat{W}_i^{CI}(L) - \widehat{W}_i^{NC}(L). \quad (17.9)$$

The next theorem considers that at most $(m - 1)$ tasks contribute at amount $\widehat{W}_i^{CI}$, and the remaining $(n - m + 1)$ ones must contribute $\widehat{W}_i^{NC}$.

Theorem 17.4 *An upper bound on the response time of a task τ_k in a multiprocessor system scheduled with a global work-conserving scheduler can be derived by the fixed point iteration on the value R_k^{ub} of the following expression, starting with $R_k^{\text{ub}} = C_k$:*

$$R_k^{\text{ub}} \leftarrow C_k + \left\lfloor \frac{1}{m} \left(\sum_{\tau_i \in \tau} \widehat{W}_i^{NC}(L) + \sum_{i|(m-1) \text{ largest}} \widehat{W}_i^\epsilon(L) \right) \right\rfloor, \quad (17.10)$$

where $\widehat{W}_i^{NC}(L)$ and $\widehat{W}_i^\epsilon(L)$ are given by Eq. 17.8 and 17.9, respectively.

As the sum of the interfering contribution is extended to all tasks, the above theorem is incomparable with the [RTA] test for work-conserving schedulers given by Theorem 17.2, as well as with the [RTA] test for systems scheduled with global EDF given by Theorem 17.3.

Sources

The response-time analysis for global work-conserving schedulers and EDF has been presented in [58]. The reduction of the carry-in instances to consider, presented in Sect. 17.1.4, is from [123].

Chapter 18

Global Fixed-Task-Priority Scheduling

We now describe some of the main schedulability tests for 3-parameter sporadic task systems that are globally scheduled using fixed-task-priority (FTP) scheduling algorithms. It will become evident that many of these tests are derived from the tests presented in the previous chapters for general work-conserving schedulers.

We start out presenting the FTP versions of the [BCL] (Sect. 15.1) and [RTA] (Sect. 17.1) tests. Then, we show that the idea described in Sect. 17.1.4 for reducing the number of carry-in instances that must be considered, is more effective when applied to the schedulability analysis of FTP scheduling algorithms.

Finally, we address the priority assignment problem and discuss a strategy that makes use of the presented schedulability tests.

Before presenting the tests, we make an important observation concerning the interference (see Sect. 14.4) imposed by lower-priority tasks in FTP schedules.

Observation 18.1 In FTP scheduling no task suffers interference from any lower-priority task. Hence

$$I_{i,k} = 0 \quad \forall i, k \mid \tau_i \text{ has lower priority than } \tau_k$$

Assuming tasks are indexed by decreasing priorities ($i > k \Rightarrow \tau_k$ has scheduling priority over τ_i), Lemma 14.1 then specializes as follows.

Theorem 18.1 *The interference on a constrained deadline task τ_k is the sum of the interferences of all higher-priority tasks divided by the number of processors:*

$$I_k = \frac{\sum_{i < k} I_{i,k}}{m}. \quad (18.1)$$

18.1 The [BCL] Test for FTP Algorithms

The “workload versus demand” strategy adopted in the [BCL] test presented in Sect. 15.1 can be applied also in the FTP case. However, a different upper bound on the interference of each task τ_i on the problem job of τ_k needs to be provided. While no carry-out job interferes with the problem job in EDF, this is not the case with a

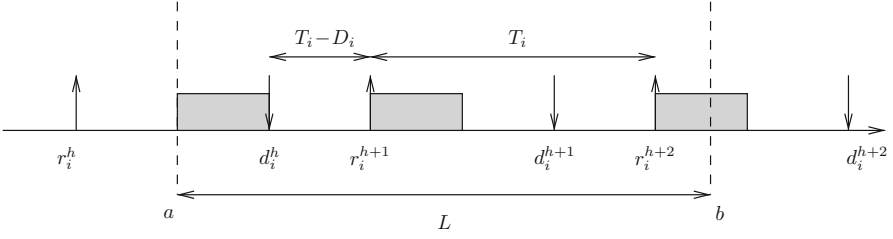


Fig. 18.1 Response-time analysis for FTP systems

general FTP scheduler. A different situation than the one depicted in Fig. 15.1 needs to be considered.

A bound on the workload of a task τ_i in the problem job's scheduling window $[r_k, r_k + D_k]$ can be computed by considering a situation in which the carry-in job starts executing as close as possible to its deadline and right at the beginning of the interval of interest, and every other instance of τ_i is executed as soon as possible. The situation is represented in Fig. 18.1. The analysis is similar to that presented for deriving Eq. 17.3 for the [RTA] test.

The first job of τ_i after the carry-in job is released at time $(r_k + C_i + T_i - D_i)$. The next jobs are then released periodically every T_i time units. Therefore the number of body jobs within the interval is

$$\left\lfloor \frac{(r_k + D_k) - (r_k + C_i - D_i + T_i)}{T_i} \right\rfloor = \left\lfloor \frac{D_k + D_i - C_i - T_i}{T_i} \right\rfloor.$$

Adding the (full) carry-in contribution, the number of τ_i 's jobs that contribute with an entire Worst-case Execution Times (WCET) to the workload in an interval of length D_k is

$$\left\lfloor \frac{D_k + D_i - C_i}{T_i} \right\rfloor.$$

The contribution of the carry-out job can then be bounded by

$$\min(C_i, (D_k + D_i - C_i) \bmod T_i).$$

A bound on the workload of a task τ_i in the problem job's scheduling window is then:

$$\left\lfloor \frac{D_k + D_i - C_i}{T_i} \right\rfloor C_i + \min(C_i, (D_k + D_i - C_i) \bmod T_i). \quad (18.2)$$

While this bound is weaker than that for EDF given by Eq. (15.2), the performance of the [BCL] test for FTP systems is significantly improved by applying Theorem 18.1.

The FTP analog of the [BCL] Theorem 15.2 is given by the following theorem.

Theorem 18.2 ([BCL] for FTP) *Task system τ is FTP schedulable if, for each $\tau_k \in \tau$,*

$$\sum_{i < k} \min \left(D_k - C_k, \left\lfloor \frac{D_k + D_i - C_i}{T_i} \right\rfloor C_i + \min(C_i, (D_k + D_i - C_i) \bmod T_i) \right) < m(D_k - C_k). \quad (18.3)$$

As in the EDF case, the schedulability test consists of n inequalities, each a sum of n terms. The computational complexity is therefore polynomial, at $O(n^2)$.

18.2 The [RTA] Test for FTP Scheduling

It is possible to similarly specialize the [RTA] test [58] to FTP-scheduled systems by applying Theorem 18.1, as the result of Theorem 17.1 can be specialized to FTP-scheduled systems by eliminating consideration of interference by all lower-priority tasks.

Theorem 18.3 *A task τ_k scheduled with FTP has a response time bounded by R_k^{ub} if*

$$\sum_{i < k} \min(I_{i,k}, R_k^{\text{ub}} - C_k + 1) < m(R_k^{\text{ub}} - C_k + 1)$$

The bound $\widehat{W}_i(L)$ on the generic workload in a window of length L given by Eq. 17.3 is still valid. It is then possible to compute an upper bound on the response time of a task τ_k as follows.

Theorem 18.4 [[RTA] for FTP] *An upper bound on the response time of a task τ_k in a multiprocessor system scheduled with FTP can be derived by the fixed point iteration on the value R_k^{ub} of the following expression, starting with $R_k^{\text{ub}} = C_k$:*

$$R_k^{\text{ub}} \leftarrow C_k + \left\lfloor \frac{1}{m} \sum_{i < k} \min(\widehat{W}_i(R_k^{\text{ub}}), R_k^{\text{ub}} - C_k + 1) \right\rfloor, \quad (18.4)$$

where $\widehat{W}_i(R_k)$ is given by Eq. 17.3.

The iterative schedulability test previously described for EDF and general work-conserving schedulers in Figs. 17.2 and 17.3 can be simplified for systems scheduled with FTP. By updating the response time upper bounds in priority order, the test can be stopped after the first round of updates. Indeed, tighter estimations of the interferences produced by lower priority tasks cannot possibly produce an improvement in the response time bound of a higher-priority task.

This observation enables the limiting of the number of updates to one for each task. Therefore, a simpler strategy can be adopted, as reported in Fig. 18.1 with procedure [SCHEDCHECKFP](τ). Function COMPUTERESPONSE(τ_k) returns the response time bound computed using Theorem 18.4.

As in the EDF case, the [RTA] test dominates the [BCL] test. However, the complexity of the [RTA] test for FTP systems is smaller than in the EDF and general cases. As mentioned in Sect. 17.1.3, computing a response time bound for a task takes $O(nD_{\max})$. Since there is only one round of slack updates for n tasks, the overall complexity is $O(n^2D_{\max})$, instead of $O(n^3D_{\max}^2)$. This reduced complexity, along with the improvement resulting from being able to neglect the interference of lower-priority tasks, makes the [RTA] particularly suitable for the schedulability analysis of FTP-scheduled systems. In fact, schedulability experiments on randomly generated workloads [55, 58] seem to indicate that procedure [SCHEDCHECKFP] outperforms the corresponding procedure [SCHEDCHECKFP] for EDF-scheduled systems, in terms of the number of schedulable task sets detected among randomly generated workloads.

18.3 Reducing the Number of Carry-in Instances

In Sect. 17.1.4, we showed how the number of carry-in tasks that need to be considered in the computation of interfering workload can be reduced to $m - 1$ for general work-conserving schedulers. A similar argument is applicable to FTP-scheduled systems, with an additional improvement: besides ignoring the interference from lower priority tasks, there is no need to consider previous jobs of τ_k in the computation of the workload interfering with the problem job of τ_k .

Theorem 18.5 (*Improved [RTA] for FTP*) *An upper bound on the worst-case response time R_k of a task τ_k in a multiprocessor system scheduled with global FTP can be derived by the fixed point iteration on the value R_k^{ub} of the following expression, starting with $R_k^{\text{ub}} = C_k$:*

$$R_k^{\text{ub}} \leftarrow C_k + \left\lfloor \frac{1}{m} \left(\sum_{i < k} \widehat{W}_i^{NC}(L) + \sum_{i < k(m-1) \text{ largest}} \widehat{W}_i^{\epsilon}(L) \right) \right\rfloor, \quad (18.5)$$

where $\widehat{W}_i^{NC}(L)$ and $\widehat{W}_i^{\epsilon}(L)$ are given by Eqs. (17.8) and (17.9), respectively.

Proof Consider an interval of interest that starts at the last time-instant, prior to the arrival of the problem job of τ_k , at which some processor is idle or is executing a task with priority lower than or equal to τ_k . Let t_o denote this time-instant.

Note that as long as $R_k \leq T_k$, the previous instances of the problem job do not contribute to the interfering workload in the considered interval. Also, from t_o to the release of the problem job, all processors are busy executing higher-priority jobs. Therefore the response time of the problem job cannot decrease if we were to shift all τ_k 's instances to the left (i.e., earlier in time), so that the release of the problem job coincides with t_o .

By the definition of t_o , there are at most $(m - 1)$ carry-in tasks interfering with τ_k . An upper bound on the response time of τ_k can then be obtained by considering a situation in which there are only $m - 1$ carry-in instances.

The theorem follows by considering that at most $(m-1)$ tasks contribute at amount $\widehat{W}_i^{CI}$, given by Eq. (17.7), and the remaining $(n-m+1)$ ones must contribute $\widehat{W}_i^{NC}$. $\square$

Note that there is no need to consider the case $i = k$ in the terms $\widehat{W}_i^{NC}(L)$ and $\widehat{W}_i^{CI}(L)$ given by Eqs. (17.8) and (17.7).

Differently from the EDF and general work-conserving cases, the above theorem dominates its original [RTA] counterpart given by Theorem 18.4, which considers the full carry-in contribution $\widehat{W}_i^{CI}$ for all tasks. However, when the number of tasks is $n \leq 2m$, both theorems are equivalent [74].

Lemma 18.1 *The response time upper bounds of the m highest-priority tasks computed with Theorem 18.4 and 18.5 are equal to their worst-case execution times:*

$$R_k^{\text{ub}} = C_k, \forall 1 \leq k \leq m.$$

Proof This can be seen by considering Eqs. (18.4) and 18.5 starting with $R_k^{\text{ub}} = C_k$. The bounds $\widehat{W}_i^{NC}(L)$ and $\widehat{W}_i^{CI}(L)$ given by Eqs. (17.8) and (17.7) are limited by the $R_k^{\text{ub}} - C_k + 1$ term in the minimum, which is equal to 1. Since there are at most $m-1$ higher priority tasks, the floor function in Eqs. (18.4) and 18.5 gives $\lfloor (m-1)/m \rfloor = 0$. Hence, the fixed point iteration immediately terminates, returning a value of $R_k^{\text{ub}} = C_k, \forall 1 \leq i \leq m$. $\square$

Theorem 18.6 *Theorem 18.4 and 18.5 are equivalent when $n \leq 2m$.*

Proof From Lemma 18.1, we know that both theorems are equivalent when $n \leq m$. Now, consider the tasks with priorities from $m+1$ to $2m$. The interference from the m highest priority tasks τ_i can be computed considering that $\forall 1 \leq i \leq m : R_i^{\text{ub}} = C_i$. In these conditions, the term $\widehat{W}_i^{CI}(L)$ given by Eq. (17.7) is equal to the corresponding term $\widehat{W}_i^{NC}(L)$ given by Eqs. (17.8), for all $1 \leq i \leq m$, so that $\widehat{W}_i^{\epsilon}(L) = 0$ for all $1 \leq i \leq m$. Out of the first $2m-1$ tasks, at most $2m-1-m = m-1$ of them can have $\widehat{W}_i^{\epsilon}(L) \geq 0$. Limiting the sum of the $\widehat{W}_i^{\epsilon}(L)$ terms in Eq. 18.5 to the largest $m-1$ contributions has therefore no effect for each of the $2m$ higher-priority tasks. Hence, Eq. 18.5 reduces to Equation (18.4), proving the theorem. $\square$

18.3.1 Reducing the Number of Carry-in Instances for [BCL]

The idea explored above for limiting the contribution of carry-in in the [RTA] test is also applicable to the [BCL] test presented in Sect. 18.1; the proof is identical to that of Theorem 18.5.

Theorem 18.7 *(Improved [BCL] for FTP) Task system τ is FTP schedulable if, for each $\tau_k \in \tau$,*

$$\left(\sum_{i < k} \widehat{W}_{i,k}^{NC} + \sum_{i < k \mid (m-1) \text{ largest}} \widehat{W}_{i,k}^{\epsilon} \right) < m(D_k - C_k), \quad (18.6)$$